

0.20. Voir x à l'écran : nombre

Le 0.19, plus `nombre(0, 2, x)` après chaque trajet : on VOIT la valeur de x changer, 005 puis 000.

1. Ce qu'il faut comprendre

C'est le 0.19, plus deux lignes identiques : `nombre(0, 2, x);` après chaque trajet.

Ce qui est nouveau ici : `nombre(colonne, ligne, valeur)`. Elle écrit un nombre à l'écran, à la case (colonne, ligne), en trois chiffres : 5 s'écrit `005`, 0 s'écrit `000`.

À quoi ça sert : une variable ne se voit pas. `x` change dans la mémoire de la console, mais rien ne le montre. En l'écrivant à l'écran, on voit ce que vaut `x` : `005` après l'aller, `000` après le retour.

Pourquoi la ligne 2 : la lettre bouge sur la ligne 0 ; le nombre, deux lignes plus bas, ne la gêne pas.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <deplace_x>` → [0.17.3. La fonction deplace x\(\) — avancer sur une ligne](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1145 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      63 o ← la console, à l'allumage - la mise en route de la console
deplace_x()    158 o ← main() - une fonction fournie par le langage
main()         69 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()      x 2   26 o sur place, dans deplace_x() + AdresseCase (10 o)
nombre()     x 2   20 o sur place, dans main() + EcrireNombre (33 o)
effacer()    x 1   15 o sur place, dans deplace_x() + AdresseCase (10 o), EffacerCases (12 o)
image()      x 2    6 o sur place, dans deplace_x(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
deplace_x() x 2   38 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <deplace_x> // fait avancer une tuile sur sa ligne
#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A
#include <nombre>     // écrit un nombre en chiffres
```

2. UNE VARIABLE — PAS

```
uint8_t pas = 5; // +5 : avance ; -5 : recule
```

3. UNE VARIABLE — X

```
uint8_t x = 0; // la colonne de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    x = deplace_x(x, 0, ALPHABET[0], pas); // l'aller : x vaut 5

    // La ligne ajoutée :
    //
    //  nombre(0, 2, x);
    //      | | |
    //      | | +-- valeur : ce qu'on écrit, la valeur de x
    //      | +----- ligne : 2, sous la lettre (qui est en ligne 0)
    //      +----- colonne : 0, tout à gauche
    //
    // Le nombre s'écrit en 3 chiffres : 5 → « 005 ».
    nombre(0, 2, x); // on VOIT x : 005

    x = deplace_x(x, 0, ALPHABET[0], -pas); // le retour : x vaut 0
    nombre(0, 2, x); // on VOIT x : 000

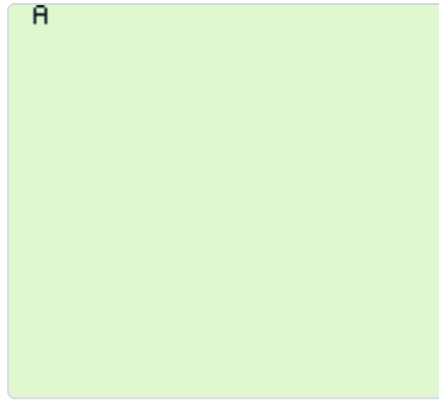
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

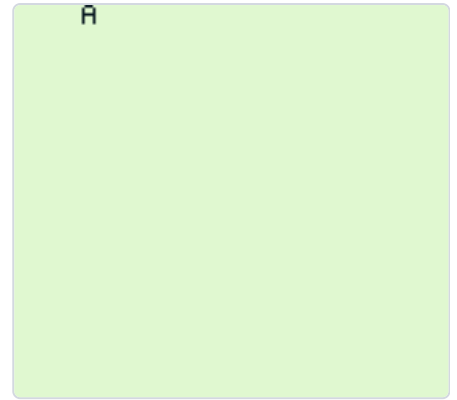
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



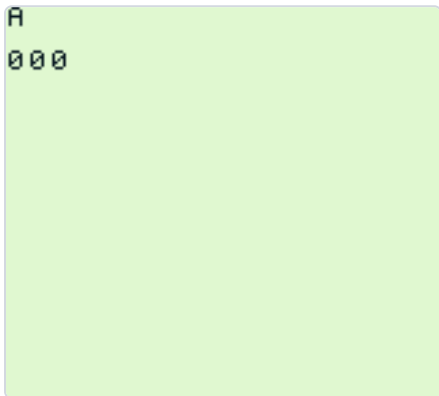
la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Le A avance de 5 cases ; en dessous s'écrit 005. Il revient ; le nombre devient 000.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ après l'aller, l'écran montre 005

✓ après le retour, 000 (« 000 »)

✓ le A est revenu en colonne 0

Ensuite : **0.20.1. Voir x à l'écran : nombre — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++